

Department of Computer and Software Engineering
SE: Machine Learning

Course Instructor:	Date:
Day:	Semester:

Lab 12: Model Deployment using Flask & Package Management

Lab Title	CLO	BT	PLO	Weightage
Model Deployment using Flask REST APIs, Docker & uv				

Name	Roll Number	Report /10	Viva /5	Total /15
Zunaira Abdul Aziz	BSSE23058			

Checked on: _____

Signature: _____

Objectives

- Convert an existing API from FastAPI to Flask
- Understand differences between API frameworks
- Deploy a trained model through a web interface and containerize it
- Utilize uv to manage modern Python environments and resolve version conflicts

Problem Statement

In the previous lab, you built a machine learning pipeline, tracked experiments using MLflow, serialized the trained model, and deployed it using a FastAPI-based REST API.

In real-world systems, different frameworks may be used depending on project requirements. Flask is a lightweight and widely used framework for building simple web applications and APIs. Furthermore, managing dependencies for complex ML packages (like MediaPipe for computer vision) often leads to version conflicts on newer Python releases.

In this lab, you will convert the FastAPI deployment into a Flask application. Following this, you will explore environment and version management using uv to resolve a simulated dependency conflict.

You are required to demonstrate the working system through screenshots of:

- The running Flask application and web interfaces
- Terminal outputs demonstrating dependency errors and their resolution

Part A: Understanding Existing API

Task 1: Analyze FastAPI Implementation

Review your Lab 11 FastAPI code and identify:

- **How the model is loaded**

The model is loaded globally upon script execution to prevent redundant disk reads.

Method: Uses `joblib.load("model.joblib")` within a try-except block.

Handling: If the file is missing, model is set to None, which triggers a 503 Service Unavailable error during prediction attempts.

- **How input is received**

Data enters the system via a POST request to the `/predict` endpoint.

Format: It expects a JSON object parsed as a Python dict containing a `features` key.

Validation: The code manually verifies the presence of the key and raises a 422 Unprocessable Entity error if it's missing.

- **How predictions are returned**

The API processes the input and returns a structured JSON response.

Transformation: The input list is converted to a NumPy array and reshaped to `(1, -1)` to satisfy the model's requirement for 2D input.

Output: The scikit-learn prediction is cast to a standard Python int to ensure JSON compatibility and returned as `{"prediction": <value>}`.

Part B: Flask Setup

Task 2: Create Flask Application

Create a Flask application with:

- A home route (/)
- A prediction route (/predict)

```
from flask import Flask, request, jsonify
import joblib
import numpy as np

app = Flask(__name__)

try:
    model = joblib.load("model.joblib")
except:
    model = None

@app.route("/", methods=["GET"])
def home():
    return jsonify({"message": "ML Model API (Flask) is running"})

@app.route("/predict", methods=["POST"])
def predict():
    # Input Reception
    data = request.get_json()

    if not data or "features" not in data:
        return jsonify({"error": "Missing 'features' key in request body"}), 422

    if model is None:
        return jsonify({"error": "Model not loaded"}), 503

    try:
        # Prediction and Return
        features = np.array(data["features"]).reshape(1, -1)
        prediction = model.predict(features)
        return jsonify({"prediction": int(prediction[0])})
    except Exception as e:
        return jsonify({"error": f"Inference error: {str(e)}"}), 400

if __name__ == "__main__":
    app.run(debug=True)
```

Part C: Model Integration

Task 3: Load and Use Model

Load the serialized model from Lab 11 and use it to make predictions.

Ensure:

- Correct input format
- Proper handling of numerical features

```
from flask import Flask, request, jsonify
import joblib
import numpy as np

app = Flask(__name__)

try:
    model = joblib.load("model.joblib")
except:
    model = None
```

```
@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()

    if not data or "features" not in data:
        return jsonify({"error": "Missing 'features' key in request body"}), 422

    if model is None:
        return jsonify({"error": "Model not loaded"}), 503

    try:
        # Prediction and Return
        features = np.array(data["features"]).reshape(1, -1)
        prediction = model.predict(features)
        return jsonify({"prediction": int(prediction[0])})
    except Exception as e:
        return jsonify({"error": f"Inference error: {str(e)}"}), 400
```

Part D: Web Interface

Task 4: Create Input Form

Create a simple HTML form to take input features from the user.

Display prediction results on the webpage.

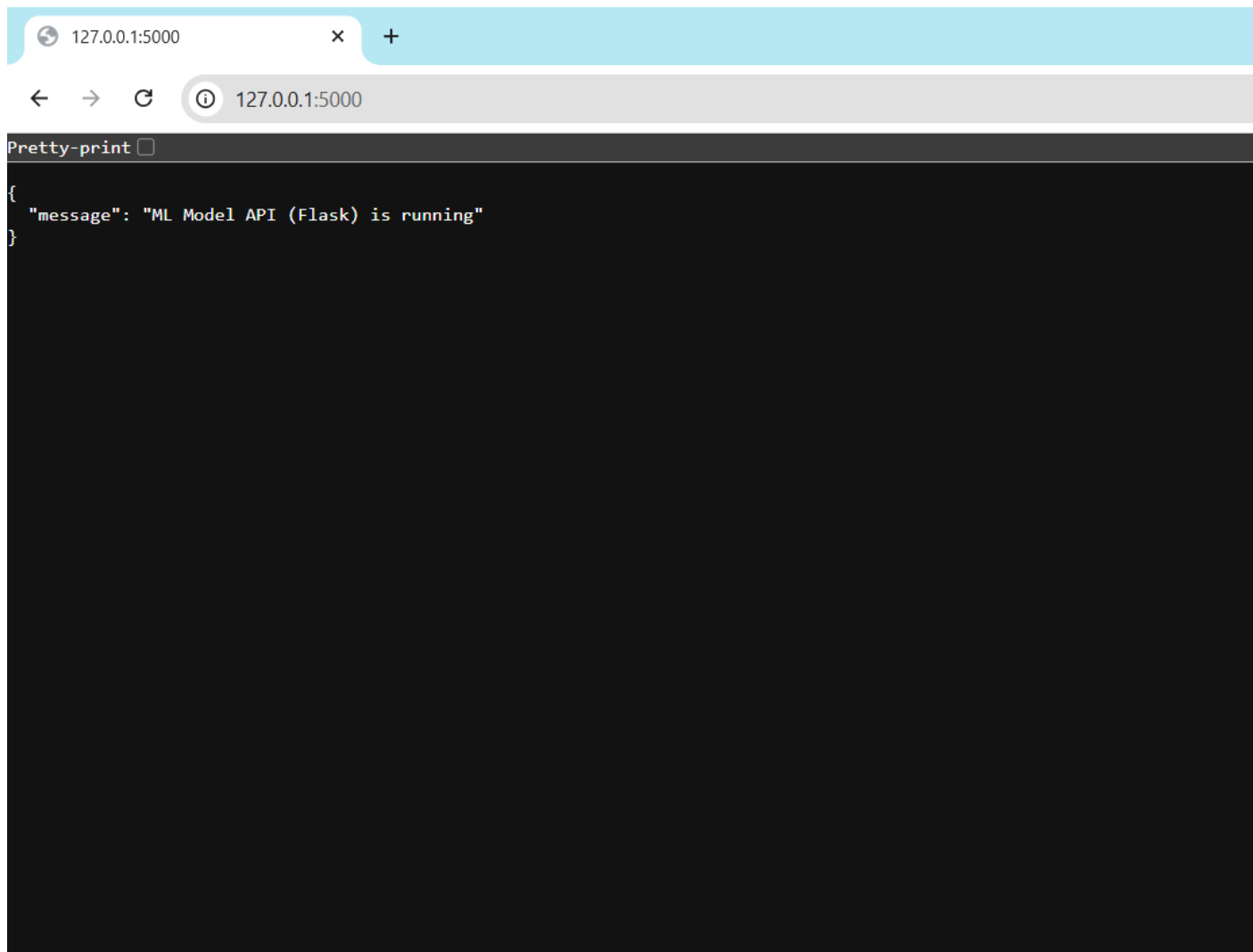
```
<!DOCTYPE html>
<html>
<head>
  <title>ML Lab 12 - BSSE23058</title>
</head>
<body>
  <h2>Enter Feature Values</h2>
  <form method="POST">
    <input type="number" step="any" name="f1" placeholder="Age" required><br><br>
    <input type="number" step="any" name="f2" placeholder="Income" required><br><br>
    <input type="number" step="any" name="f3" placeholder="Hour" required><br><br>
    <input type="number" step="any" name="f4" placeholder="Leak Feature"
required><br><br>
    <button type="submit">Predict</button>
  </form>

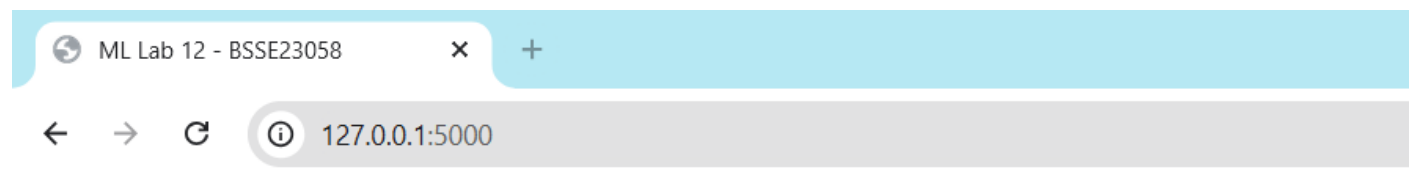
  {% if prediction is not none %}
    <h3>Result: {{ "Positive (1)" if prediction == 1 else "Negative (0)" }}</h3>
  {% endif %}
</body>
</html>
```

Task 5: Demonstration

Run the Flask application and capture screenshots of:

- Running server (terminal)
- Input form in browser
- Prediction output





Enter Feature Values

With values: 25, 5000, 14, 0.5



Enter Feature Values

Result: Negative (0)

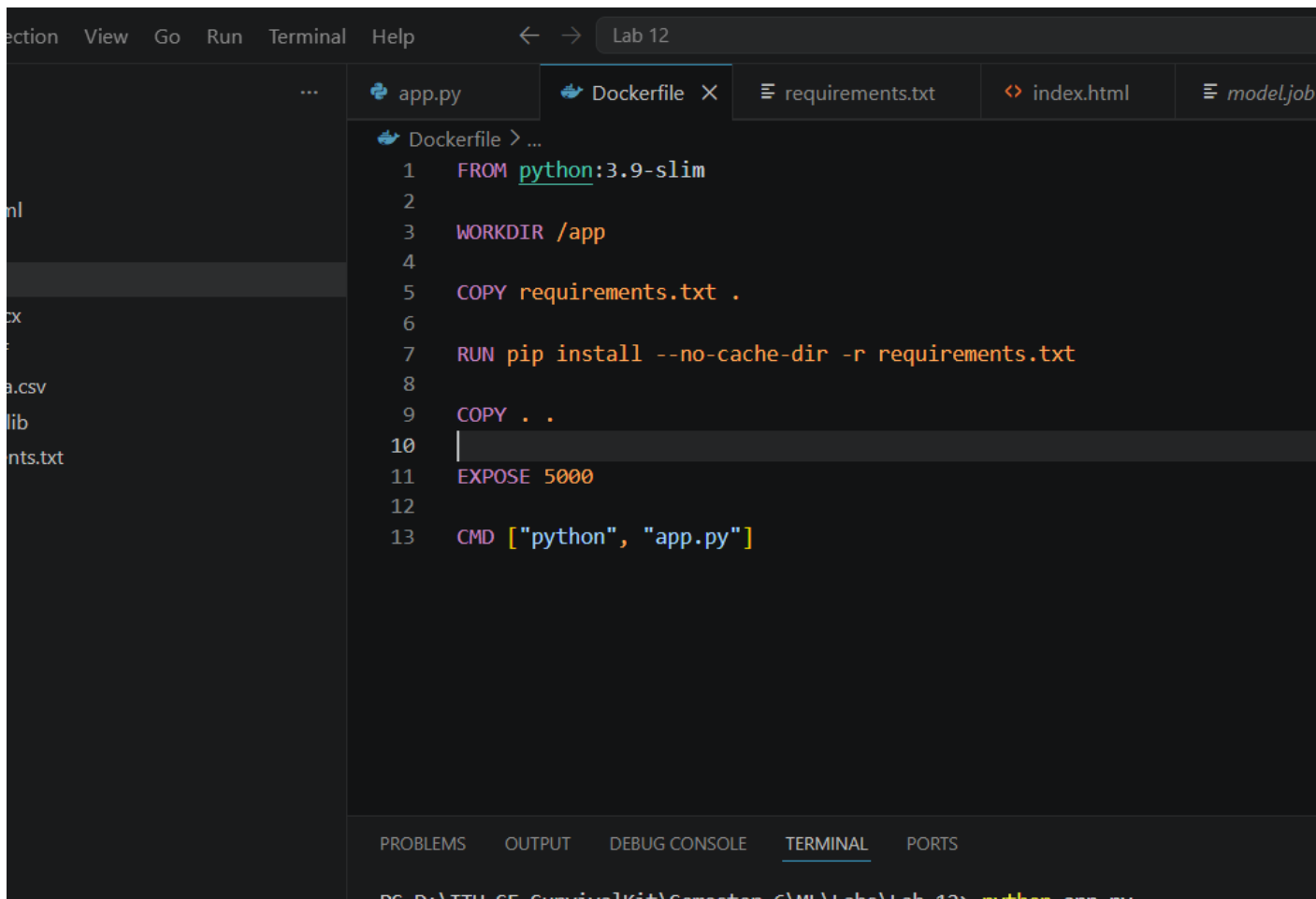
Part E: Testing and Screenshots

Task 6: Create Dockerfile

Create a Dockerfile for your Flask application.

Ensure:

- Base image uses Python
- Application dependencies are installed
- Flask app runs on port 5000

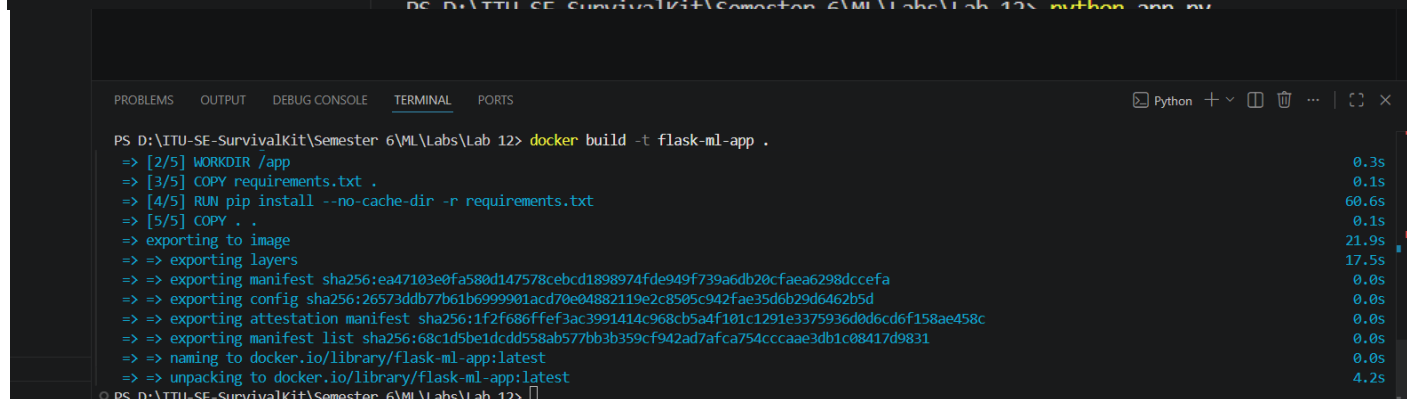


The screenshot shows the VS Code editor interface with the 'Dockerfile' tab selected. The Dockerfile content is as follows:

```
Dockerfile > ...
1 FROM python:3.9-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6
7 RUN pip install --no-cache-dir -r requirements.txt
8
9 COPY . .
10
11 EXPOSE 5000
12
13 CMD ["python", "app.py"]
```

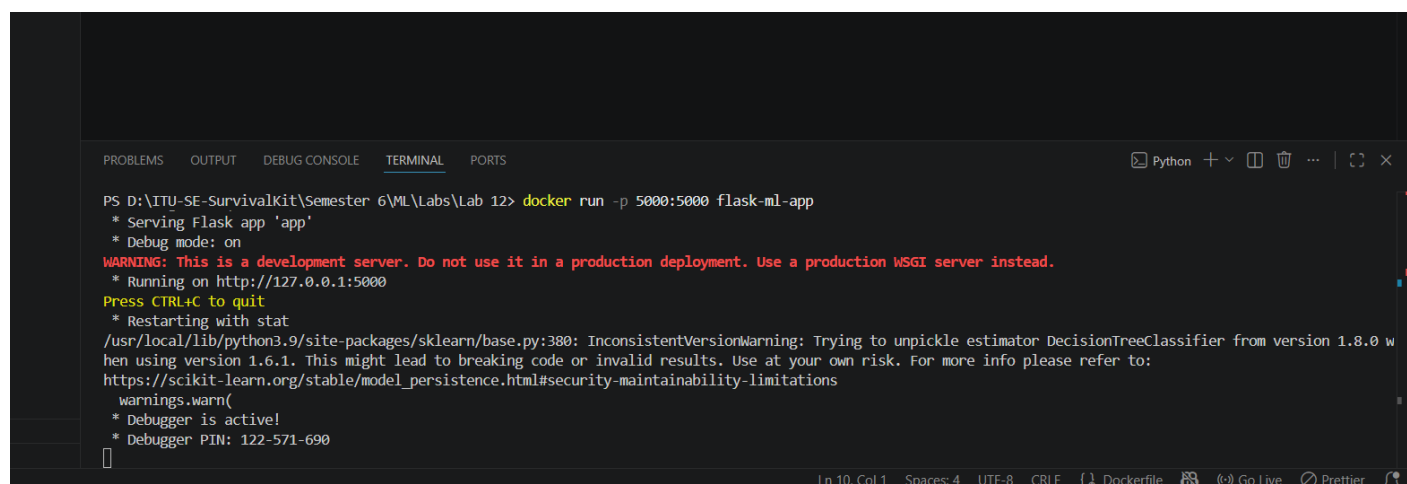
The bottom panel shows the 'TERMINAL' tab with the following output:

```
PS D:\ITU-SE-SurvivalKit\Semester 6\ML\Labs\Lab 12> python app.py
```



The screenshot shows the VS Code terminal with the output of the 'docker build' command. The output is as follows:

```
PS D:\ITU-SE-SurvivalKit\Semester 6\ML\Labs\Lab 12> docker build -t flask-ml-app .
=> [2/5] WORKDIR /app 0.3s
=> [3/5] COPY requirements.txt . 0.1s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt 60.6s
=> [5/5] COPY . . 21.9s
=> exporting to image 17.5s
=> exporting layers 0.0s
=> exporting manifest sha256:ea47103e0fa580d147578cebcd1898974fde949f739a6db20cfaea6298dccefa 0.0s
=> exporting config sha256:26573ddb77b61b6999901acd70e04882119e2c8505c942fae35d6b29d6462b5d 0.0s
=> exporting attestation manifest sha256:1f2f686ffef3ac3991414c968cb5a4f101c1291e3375936d0d6cd6f158ae458c 0.0s
=> exporting manifest list sha256:68c1d5be1dcdd558ab577bb3b359cf942ad7afca754ccaae3db1c08417d9831 0.0s
=> naming to docker.io/library/flask-ml-app:latest 0.0s
=> unpacking to docker.io/library/flask-ml-app:latest 4.2s
```



The screenshot shows the VS Code terminal with the output of the 'docker run' command. The output is as follows:

```
PS D:\ITU-SE-SurvivalKit\Semester 6\ML\Labs\Lab 12> docker run -p 5000:5000 flask-ml-app
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
/usr/local/lib/python3.9/site-packages/sklearn/base.py:380: InconsistentVersionWarning: Trying to unpickle estimator DecisionTreeClassifier from version 1.8.0 when using version 1.6.1. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
* Debugger is active!
* Debugger PIN: 122-571-690
```

Part F: Containerization using Docker

Task 7: Build and Run Docker Image

Build the Docker image and run it locally.

Verify:

- Application runs inside container
- API is accessible via browser

Screenshot Required:

- Docker build output
- Running container and browser output

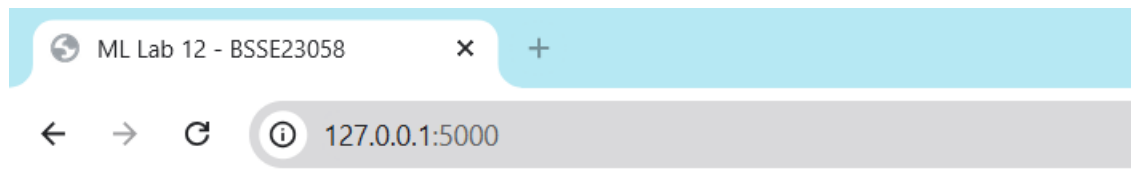
The screenshot shows the AWS CloudShell console interface. At the top, there's a navigation bar with the AWS logo and a search bar. Below it, the 'Console Home' section is visible. The main area displays the 'CloudShell' terminal window for the 'us-east-1' region. The terminal shows the following commands and output:

```

~ $ docker build -t flask-ml-app .
[+] Building 18.8s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 420B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> transferring context: 2B
=> [1/4] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b
=> => sha256:085da638e1b8a449514c3fda83ff50a3bffa4418b050cfacd87e5722071f497 5.40kB / 5.40kB
=> => sha256:38513bd7256313495cd83b3b0915a633cfae75dc2e07072ab2cd191020ca5d 29.70MB / 29.70MB
=> => sha256:b3ec39b36ae8c83a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d 1.29MB / 1.29MB
=> => sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710affa286c08 13.88MB / 13.88MB
=> => sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b 10.36kB / 10.36kB
=> => sha256:dad5b29e3506c35e0fd22736f4d4ef25d21b219acdd73f7bb41d59996ca8e0d 1.74kB / 1.74kB
=> => sha256:ea56f685404adf81680322f152d2cfec62115b30dda481c2c450078315beb508 251B / 251B
=> => extracting sha256:38513bd7256313495cd83b3b0915a633cfae75dc2e07072ab2cd191020ca5d
=> => extracting sha256:b3ec39b36ae8c83a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d
=> => extracting sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710affa286c08
=> => extracting sha256:ea56f685404adf81680322f152d2cfec62115b30dda481c2c450078315beb508
=> [internal] load build context
=> => transferring context: 9.85kB
=> [2/4] WORKDIR /app
=> [3/4] COPY . /app
=> [4/4] RUN pip install --no-cache-dir Flask numpy scikit-learn
=> exporting to image
=> exporting layers
=> writing image sha256:dc8d3d58bc030cde71922e5244f0d92afa11e11666921fc31745b639e102c8df
=> naming to docker.io/library/flask-ml-app
~ $

009100744015
~ $ docker run -d -p 5000:5000 flask-ml-app
d52f4536cd9bd883a60b1fa1ca33beb3568eb307140e2eceb967c9862cdb6445
~ $ curl http://localhost:5000
}
~ $ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS               NAMES
d52f4536cd9b   flask-ml-app   "python app.py"         14 minutes ago Up 13 minutes    0.0.0.0:5000->5000/tcp   mystifying_archimedes
~ $

```



Enter Feature Values

Part G: AWS ECR (Elastic Container Registry)

Task 8: Create ECR Repository

Create a repository in AWS ECR.

Login to ECR using AWS CLI and authenticate Docker.

Task 9: Push Docker Image to ECR

Tag and push your Docker image to ECR.

Screenshot Required:

- Successful image push
- ECR repository showing uploaded image

```
aws <command> <subcommand> help

~ $ aws ecr create-repository --repository-name flask-ml-repo
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:us-east-1:608393766790:repository/flask-ml-repo",
    "registryId": "608393766790",
    "repositoryName": "flask-ml-repo",
    "repositoryUri": "608393766790.dkr.ecr.us-east-1.amazonaws.com/flask-ml-repo",
    "createdAt": "2026-05-08T05:52:58.380000+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
~ $
```

```
Error! Cannot perform an interactive login from a non TTY device
~ $ # 1. Get your Account ID
~ $ ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
~ $
~ $ # 2. Set your Region (Change us-east-1 if yours is different)
~ $ REGION="us-east-1"
~ $
~ $ # 3. Double check they are NOT empty
~ $ echo "Account ID is: $ACCOUNT_ID"
Account ID is: 608393766790
~ $ echo "Region is: $REGION"
Region is: us-east-1
~ $
```

```
Region is: us-east-1
~ $ aws ecr get-login-password --region $REGION | docker login --username AWS --password-stdin $ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com
WARNING! Your password will be stored unencrypted in /home/cloudshell-user/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
~ $
```

```
~ $ docker push $ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/flask-ml-repo:latest
The push refers to repository [608393766790.dkr.ecr.us-east-1.amazonaws.com/flask-ml-repo]
1f85e13422c6: Pushed
ed74326d76e5: Pushed
3e86f641cf25: Pushed
c8f6b54339a8: Pushed
298992e09a03: Pushed
4f237755fbae: Pushed
d7c97cb6f1fe: Pushed
latest: digest: sha256:bc5182d6034a1187b019fcae898026f10b104d6f13a8a619872163f4d13ec2b6 size: 1785
~ $
```

Part H: Deployment using Elastic Beanstalk

Task 10: Launch Environment

Using AWS CloudShell or local CLI:

- Initialize Elastic Beanstalk application
- Configure environment (Docker platform)
- Deploy application using ECR image

Task 11: Verify Deployment

Access the deployed application via public URL.

Verify:

- Flask app is running
- Predictions are working

Screenshot Required:

- Elastic Beanstalk dashboard
- Live application URL in browser
- Prediction output

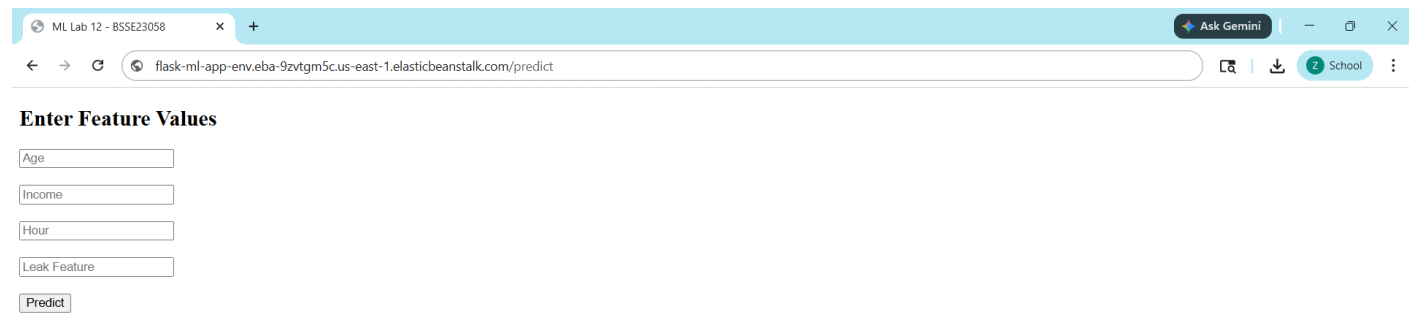
```
latest: digest: sha256:bc5182d6034a1187b019fcae898026f10b104d6f13a8a619872163f4d13ec2b6 size:
~ $ nano Dockerrun.aws.json
~ $ eb init -p docker flask-ml-app --region $REGION
Application flask-ml-app has been created.
~ $ eb create flask-env
```

{

```
"AWSEBDockerrunVersion": "1",
"Image": {
  "Name": "608393766790.dkr.ecr.us-east-1.amazonaws.com/flask-ml-repo:latest",
  "Update": "true"
},
"Ports": [
  {
    "ContainerPort": 5000
  }
]
}
```

The screenshot displays the AWS Elastic Beanstalk console, specifically the 'Environments' page. The left-hand navigation pane shows 'Elastic Beanstalk' as the selected service, with sub-links for 'Applications', 'Environments', and 'Change history'. Below this, a 'Recent environments' section lists 'Flask-ml-app-env'. The main content area, titled 'Environments (1) Info', features a search bar and a table of environments. The table has columns for 'Environment name', 'Health', 'Application', 'Platform', 'Domain', 'Running version', 'Tier name', 'Date created', and 'Last modified'. A single environment, 'Flask-ml-app-env', is listed with a 'Healthy' status (indicated by a green checkmark and 'Ok' text). The application is 'flask-ml-app', the platform is 'Docker running on Amazon Linux 2', and the domain is 'Flask-ml-app-env.eba-9zvtgm...'. The running version is 'version-1' on a 'WebServer' tier, created and last modified on May 8, 2026.

Environment name	Health	Application ...	Platform	Domain	Running ver...	Tier name	Date created	Last modified
Flask-ml-app-env	Ok	flask-ml-app	Docker running...	Flask-ml-app-env.eba-9zvtgm...	version-1	WebServer	May 8, 2026 1...	May 8, 2026 1...



The screenshot shows a web browser window with a single tab titled "ML Lab 12 - BSSE23058". The address bar displays the URL "flask-ml-app-env.eba-9zvtgm5c.us-east-1.elasticbeanstalk.com/predict". The browser's toolbar includes navigation buttons (back, forward, refresh), a search icon, and a "School" extension. The main content area of the browser displays the following form:

Enter Feature Values

Age

Income

Hour

Leak Feature